

3 - Deep Learning

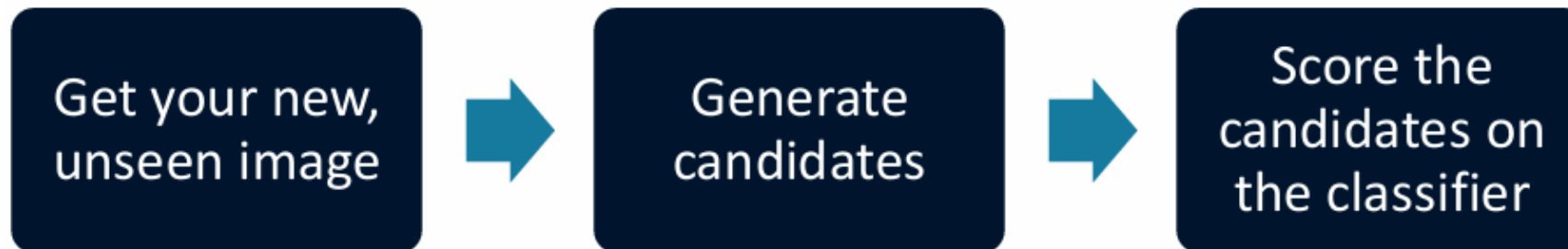
Convolutional Neural Networks, Object Detection, Image Segmentation

Object Detection

- Object detection training:

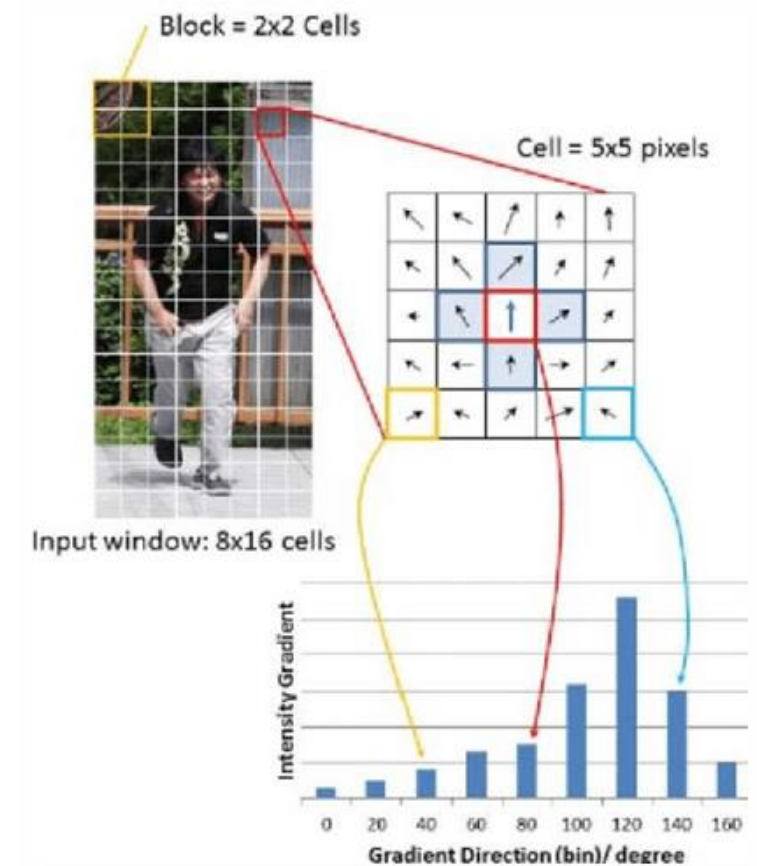


- Object detection testing:



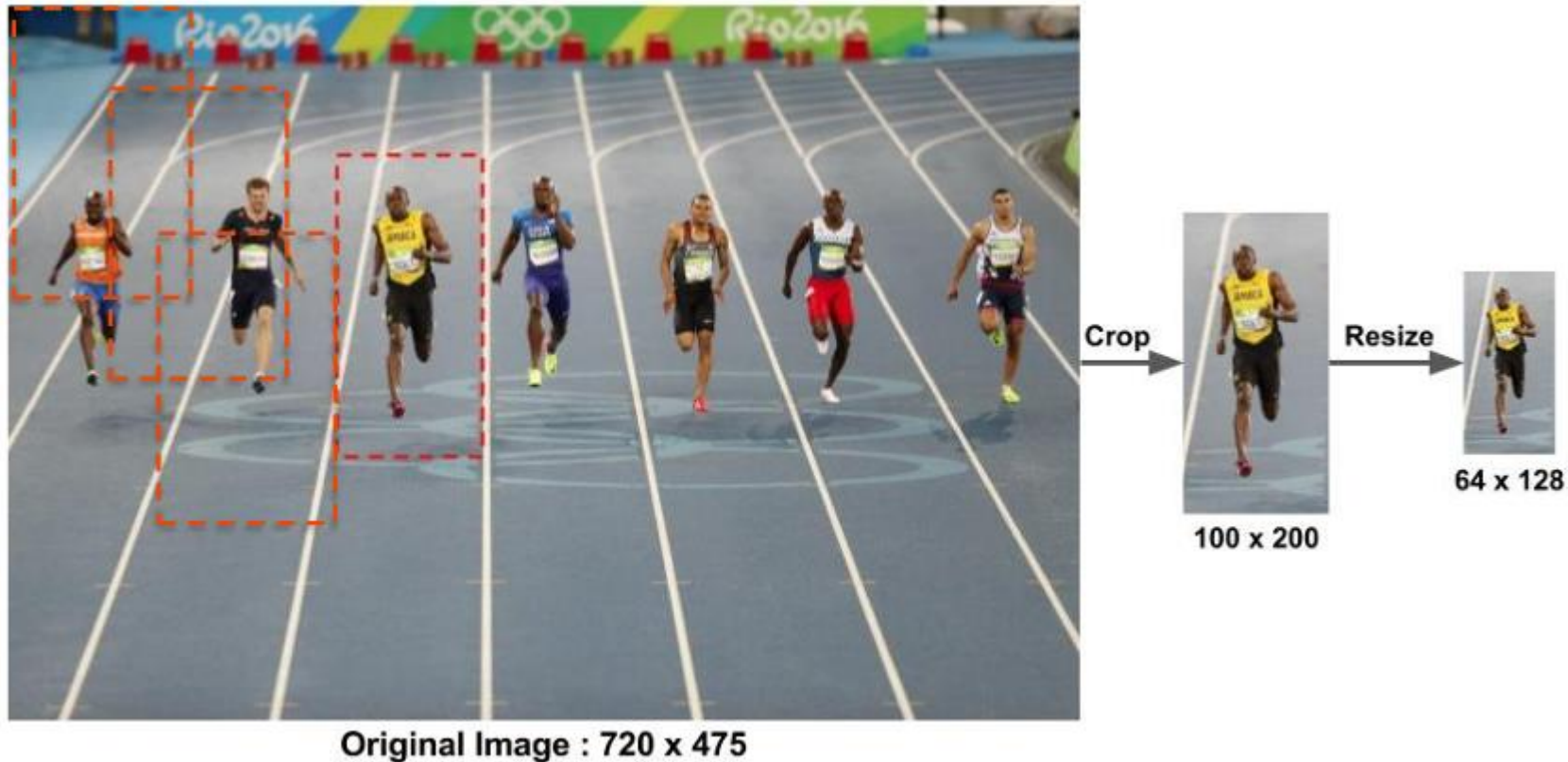
Object Detection

- Challenges in object detection: overlapping objects, viewpoint variations, background clutter, illumination changes, deformation, occlusion, etc.
- *What feature representation?*
 - Vectors of pixels? Sensitive to illumination, misalignments, etc.
 - Histograms? Sensitive to misalignments, large difference between instances, etc.
 - **Histogram of Oriented Gradients (HOG).**



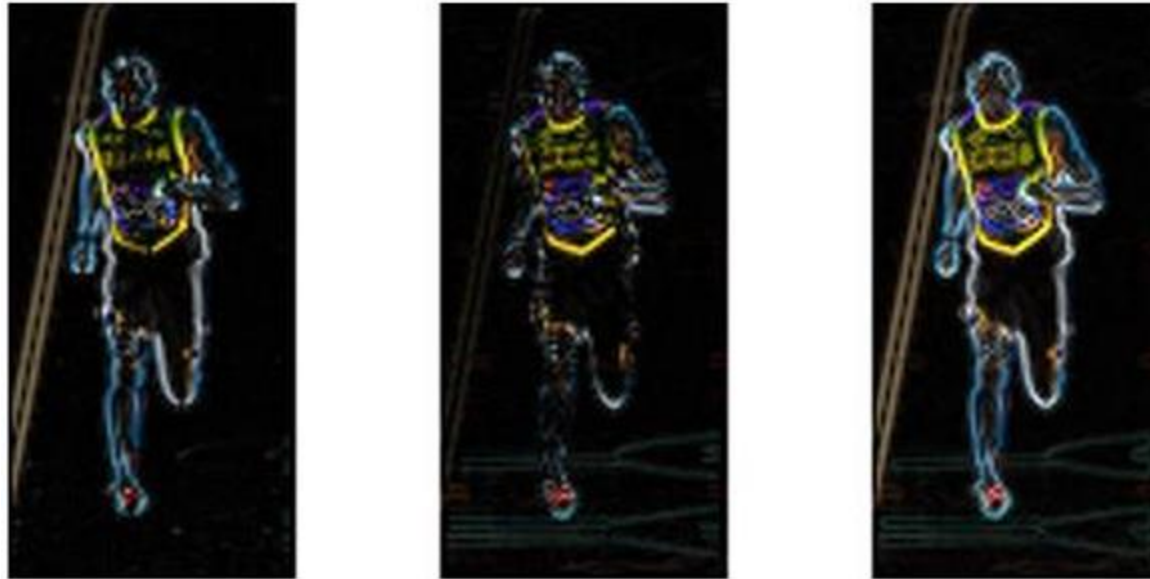
Object Detection - HOG Descriptor

1. Image preprocessing:



Object Detection - HOG Descriptor

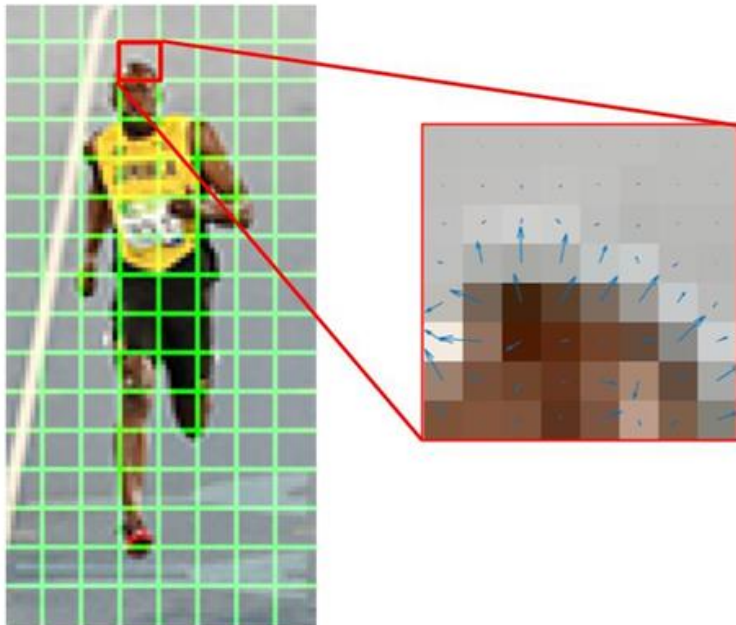
2. Compute the gradients:



Left : Absolute value of x-gradient. Center : Absolute value of y-gradient. Right : Magnitude of gradient.

Object Detection - HOG Descriptor

3. Compute histogram of gradients in 8x8 cells (weighted by magnitude):



2	3	4	4	3	4	2	2
5	11	17	13	7	9	3	4
11	21	23	27	22	17	4	6
23	99	165	135	85	32	26	2
91	155	133	136	144	152	57	28
98	196	76	38	26	60	170	51
165	60	60	27	77	85	43	136
71	13	34	23	108	27	48	110

Gradient Magnitude

80	36	5	10	0	64	90	73
37	9	9	179	78	27	169	166
87	136	173	39	102	163	152	176
76	13	1	168	159	22	125	143
120	70	14	150	145	144	145	143
58	86	119	98	100	101	133	113
30	65	157	75	78	165	145	124
11	170	91	4	110	17	133	110

Gradient Direction

80	36	5	10	0	64	90	73
37	9	9	179	78	27	169	166
87	136	173	39	102	163	152	176
76	13	1	168	159	22	125	143
120	70	14	150	145	144	145	143
58	86	119	98	100	101	133	113
30	65	157	75	78	165	145	124
11	170	91	4	110	17	133	110

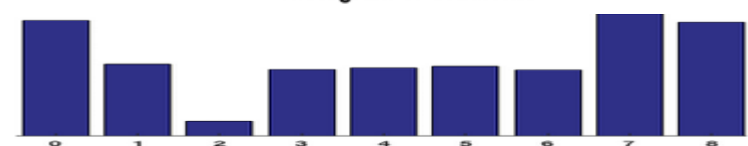
Gradient Direction

2	3	4	4	3	4	2	2
5	11	17	13	7	9	3	4
11	21	23	27	22	17	4	6
23	99	165	135	85	32	26	2
91	155	133	136	144	152	57	28
98	196	76	38	26	60	170	51
165	60	60	27	77	85	43	136
71	13	34	23	108	27	48	110

Gradient Magnitude



Histogram of Gradients



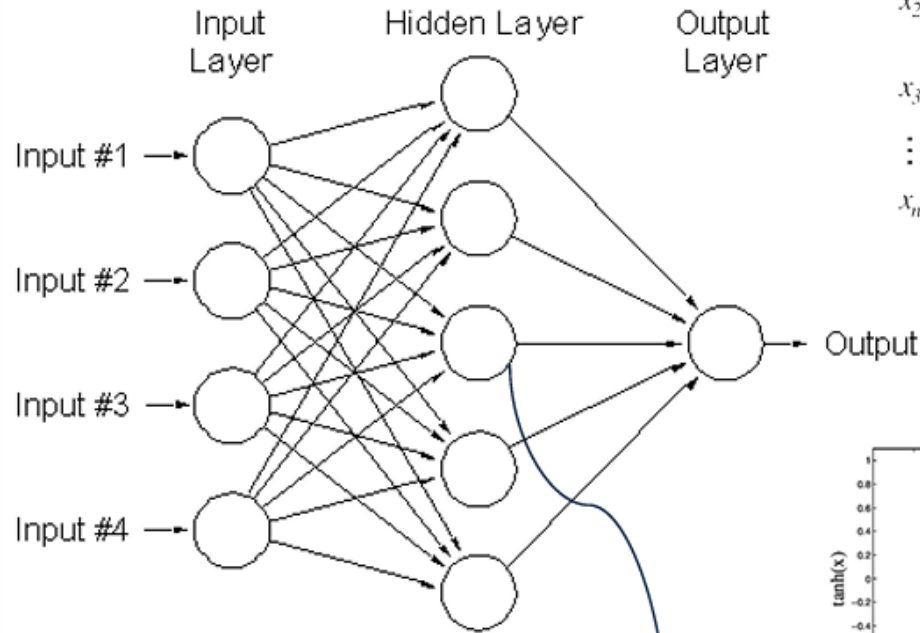
Object Detection - HOG Descriptor

4. Normalization:

- Normalize each block of 16x16 pixels (4 histograms $\rightarrow$ 1x36 feature vector, divide each element by the L2 norm).
- Concatenate all (normalized) vectors to obtain the final feature vector.
- Train any classifier on this feature space using standard ML techniques.



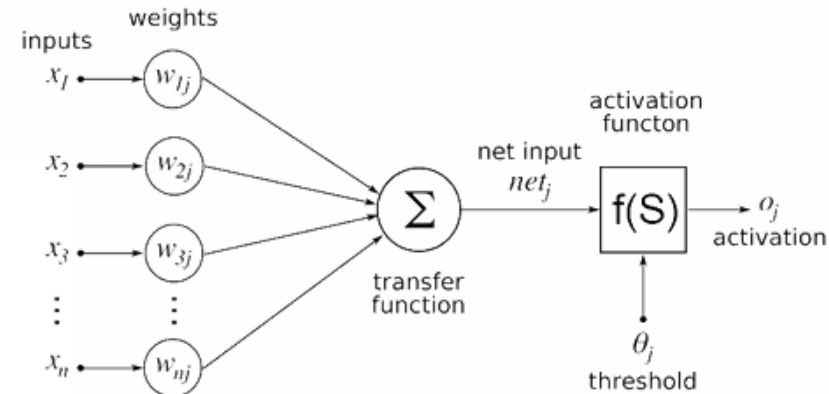
Neural Networks



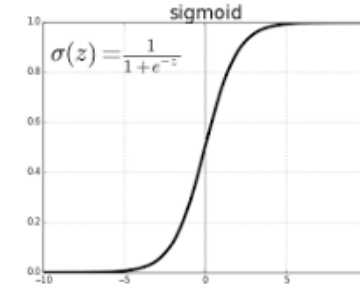
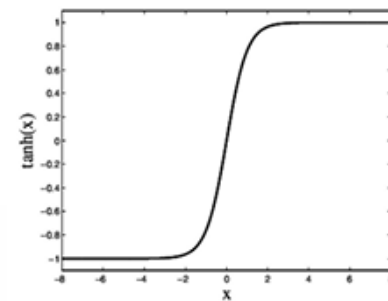
Dropout:
during training, randomly ignore activations with probability p .

Sigmoid activation function

$$\sigma\left(\sum_{i=1}^{N_{in}} w_{ji}^{(1)} x_i + b_j^{(1)}\right) = h_j$$



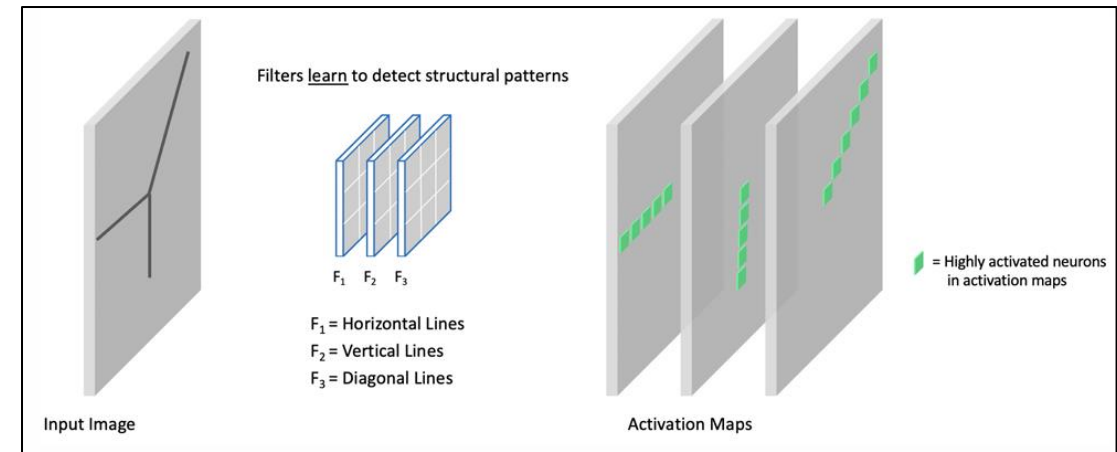
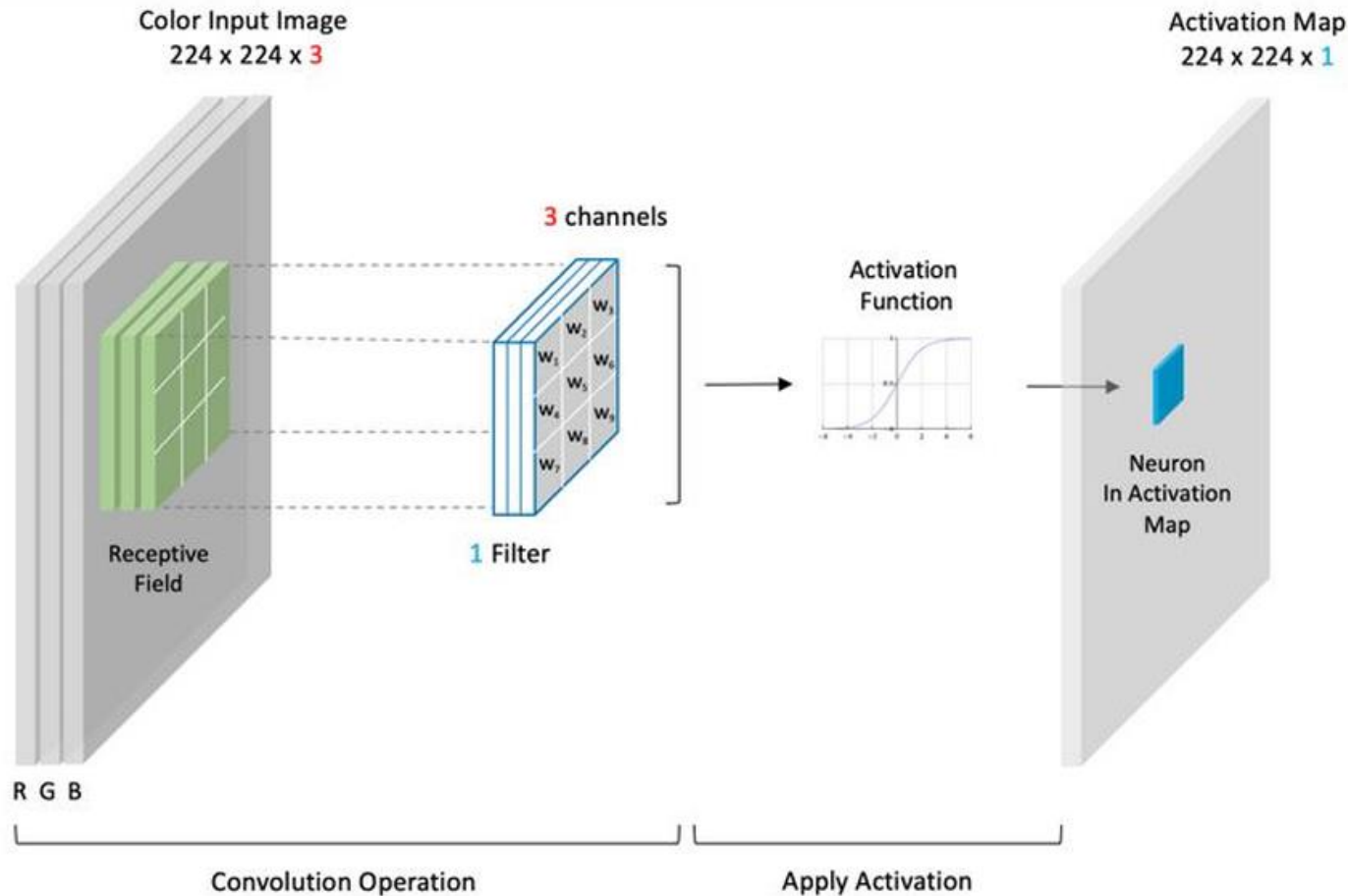
Example activation functions



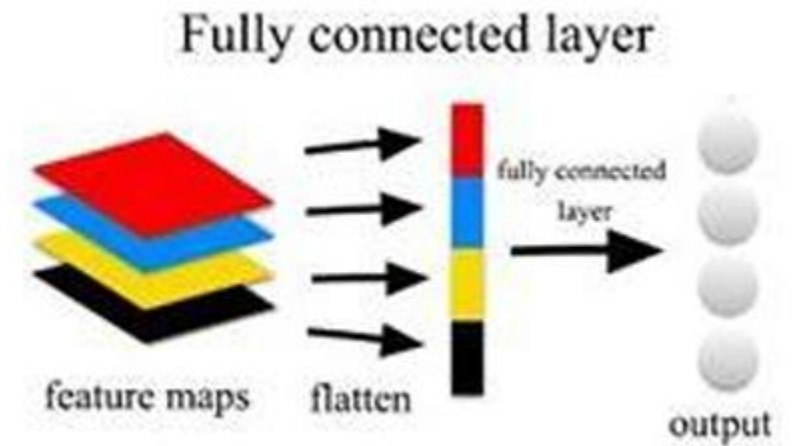
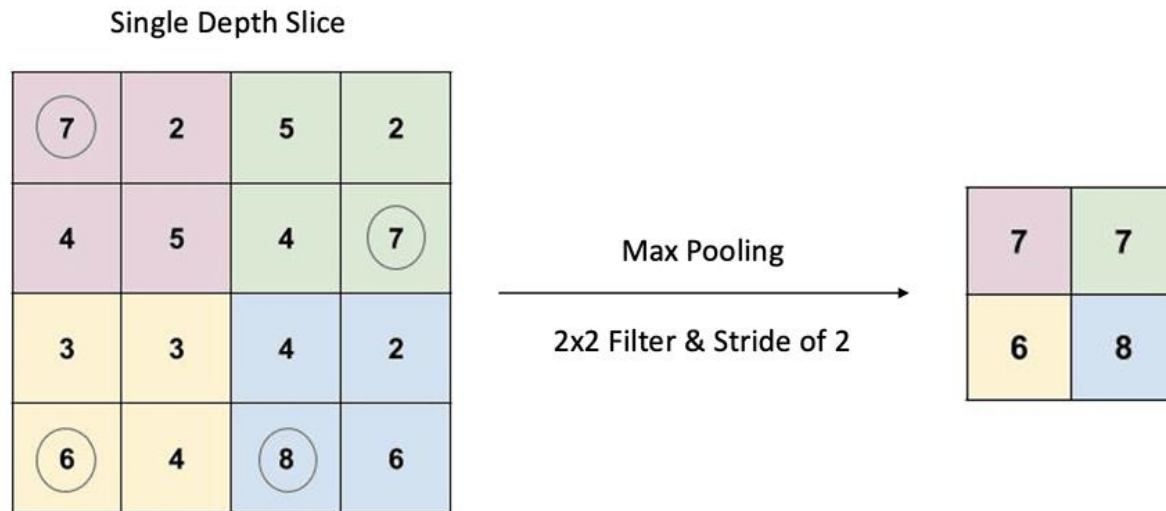
The purpose of the activation function is to introduce non-linearity into the output of a neuron.

Mini-batch SGD:
Sample a batch of data, forward it through the model, backpropagate to calculate gradients, update weights.

Convolutional Neural Networks (CNNs)

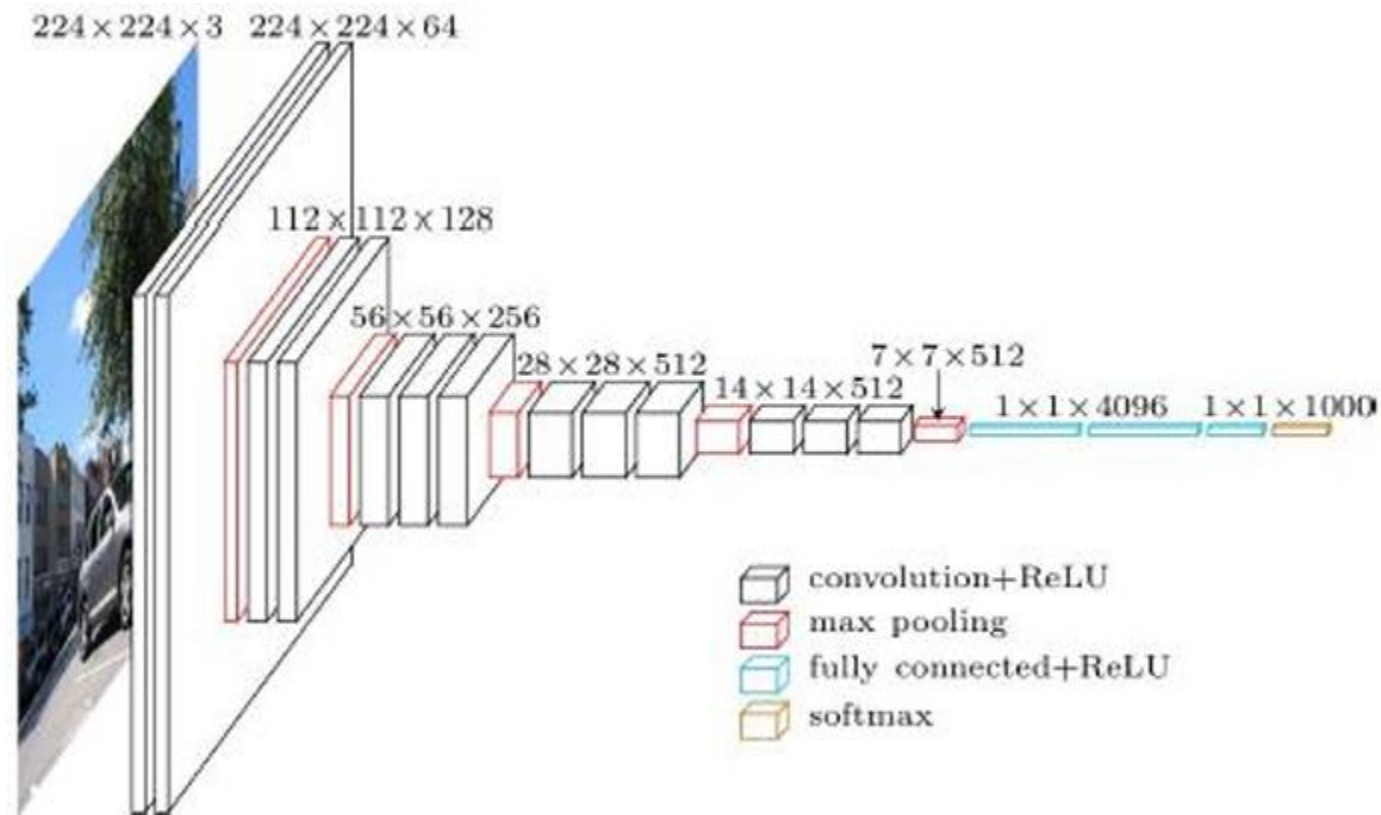


Convolutional Neural Networks (CNNs)



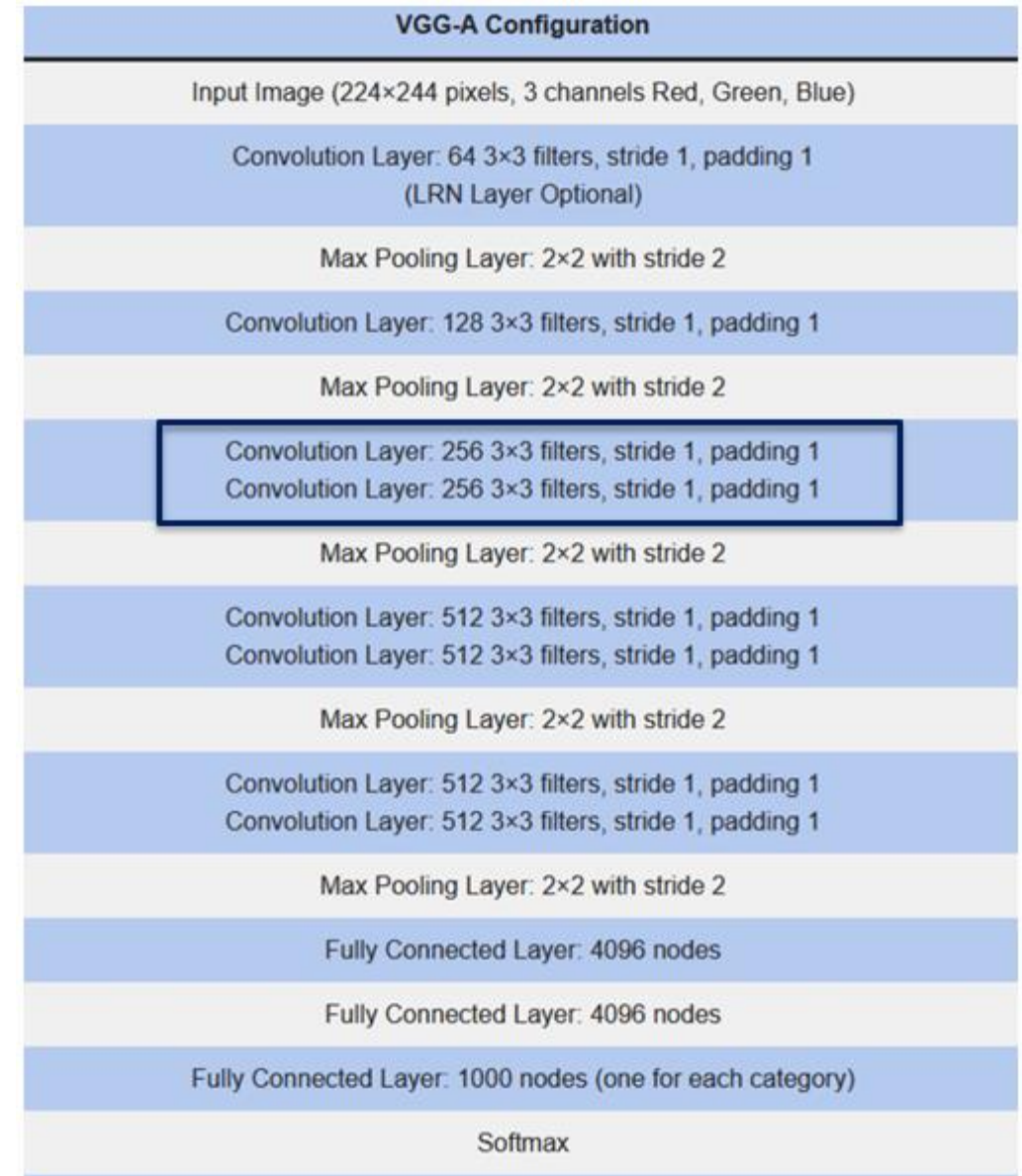
Convolutional Neural Networks (CNNs)

- Example on *ImageNet* (1.3 million images, 1000 classes):



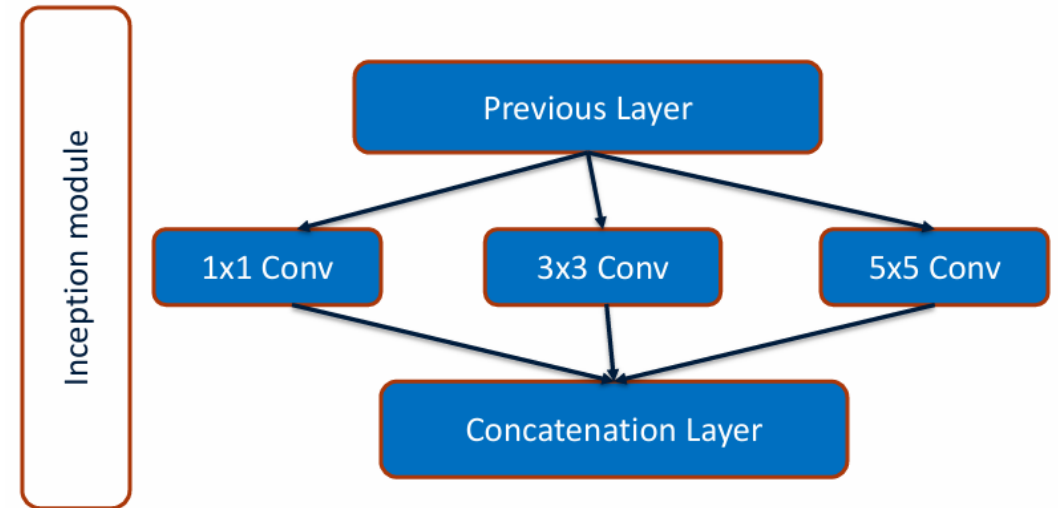
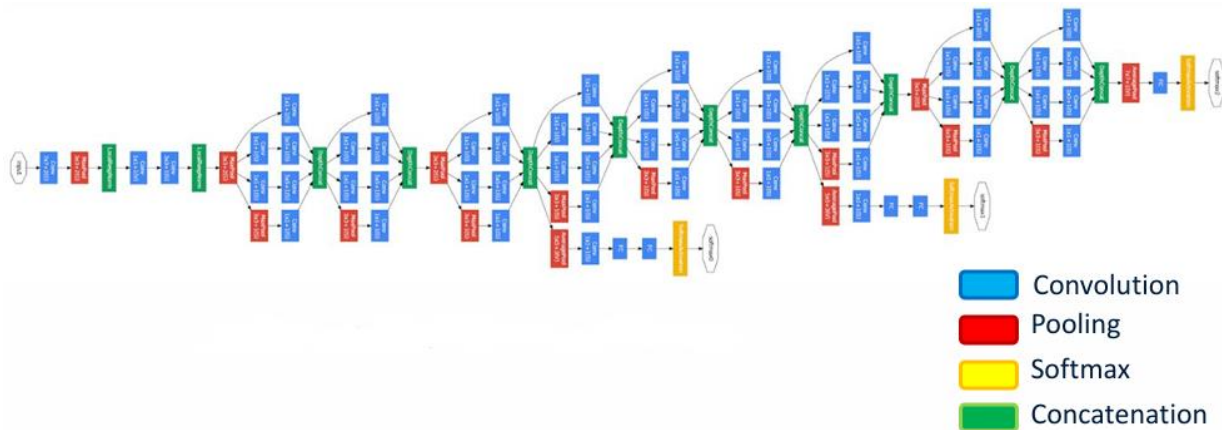
VGG Net (2014)

- In comparison to AlexNet, replaces large 11x11 and 5x5 filters with *multiple* 3x3 filters (same coverage, fewer parameters, more non-linearity).
- Great for learning due to its simplicity.
- Slow to train due to its large architecture (number of weights).



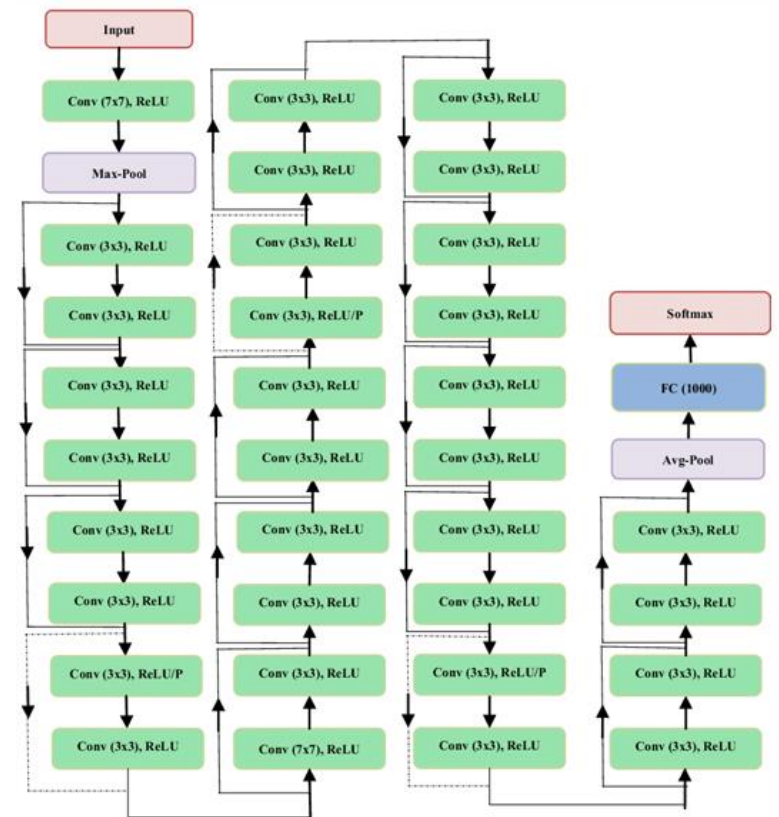
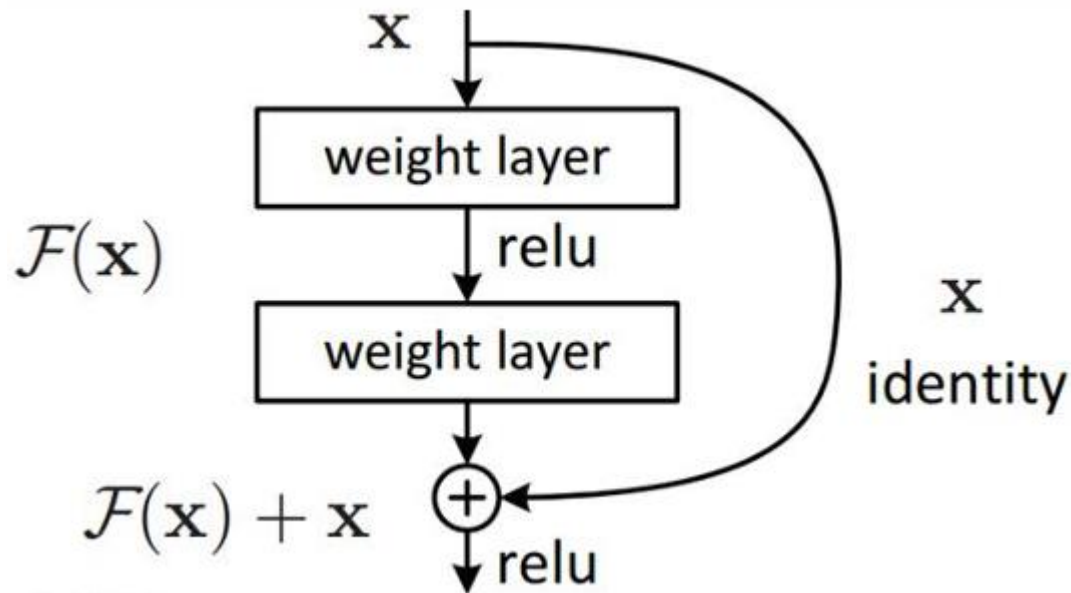
GoogLeNet (2014)

- In comparison to AlexNet, only has 4 million weights instead of 60 million.
- Introduced the **Inception Layer**: automatically learn which filter size will be useful at each level to learn relevant information.



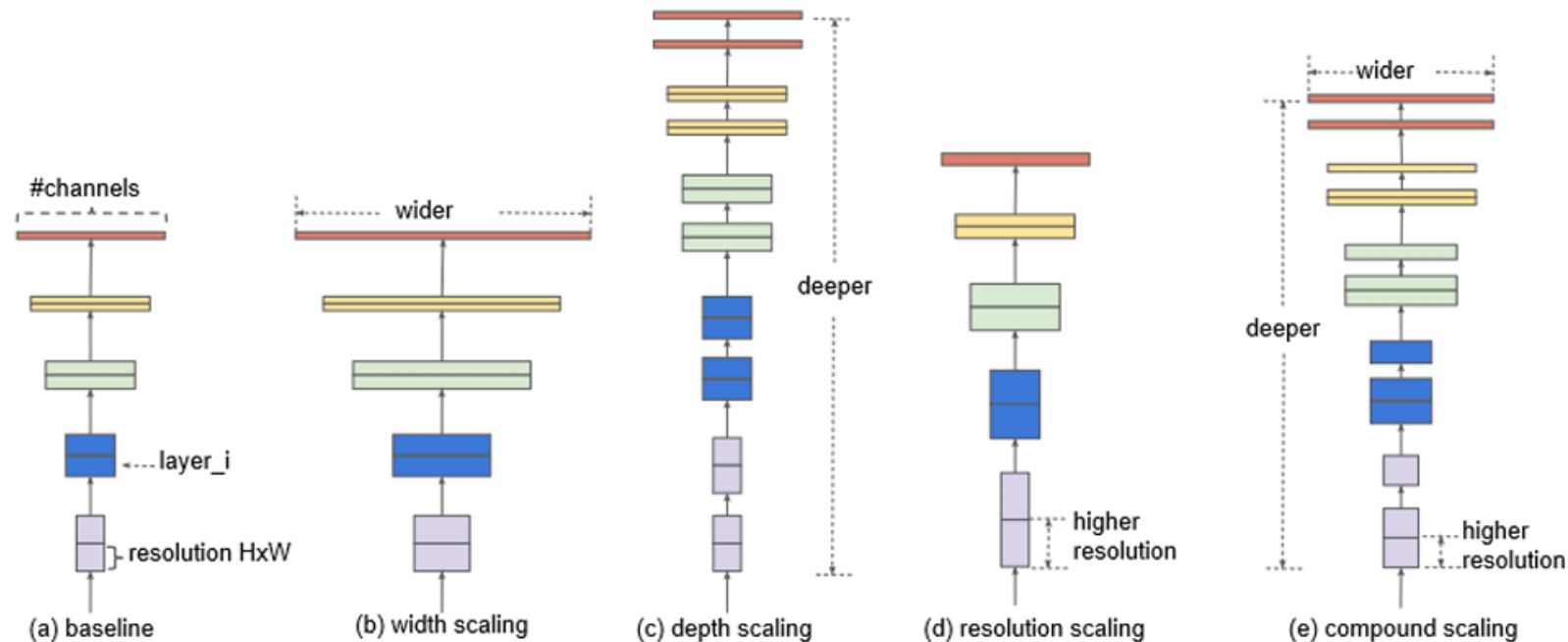
ResNet (2015)

- Very deep network (152 layers), lots of parameters.
- Introduced the **Residual Block**: input of a layer is passed to two layers later, helps with vanishing or exploding gradients.



EfficientNet (2020)

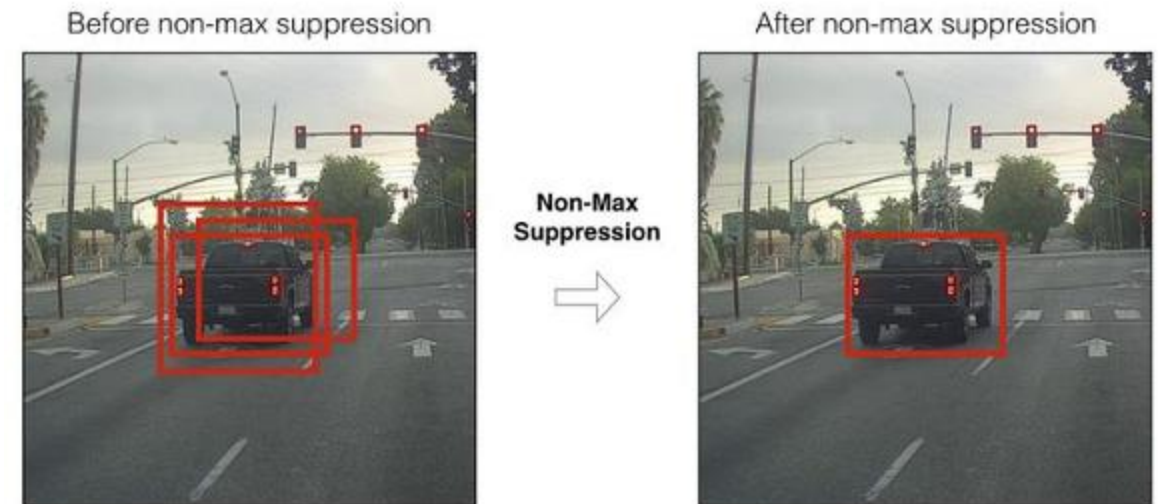
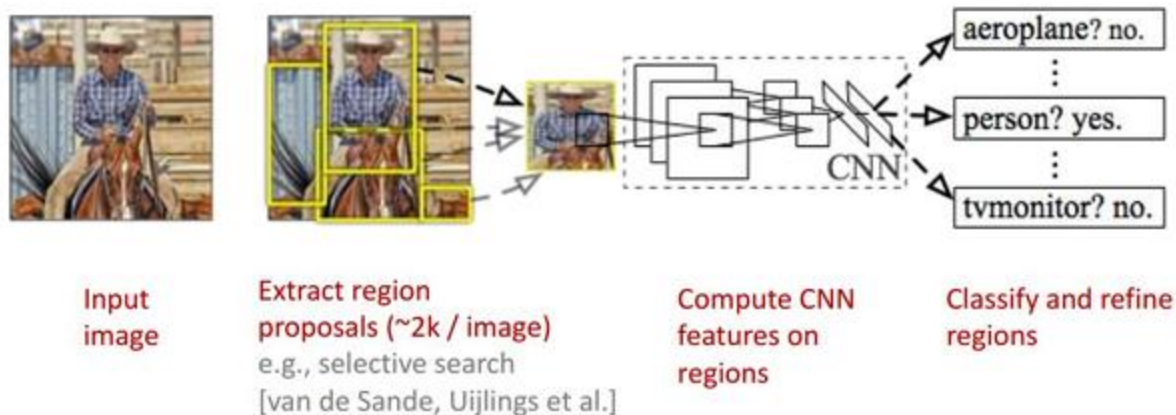
- Achieves state of the art results with significantly higher efficiency.
- Introduced the concept of **Compound Scaling**: rather than scaling in one dimension (depth, width, resolution), uniformly scale all using a *compound coefficient* ϕ .



Object Detection - R-CNN (2014)

- **Region-based Convolutional Neural Network (R-CNN):**

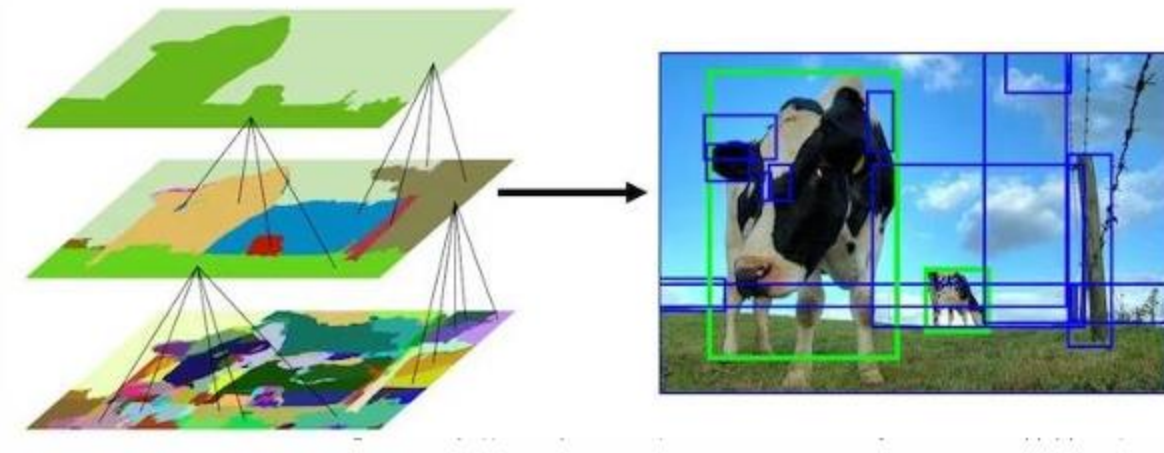
1. **Region proposal:** extract several bounding-box candidates using *selective search*.
2. Extract features from each candidate independently using a CNN (VGG, ResNet, etc.).
3. Classify features, refine the bounding box, and apply *non-maximum suppression*.



(use IoU to determine if the boxes are detecting the same object)

Object Detection - R-CNN (2014)

- **Selective Search:** identify candidate objects using *segmentation*, reducing the search considerably compared to the sliding window approach.
- Use a maximally over-segmented image as input in the first stage, then segment more based on similarity (color, texture, size, and shape).

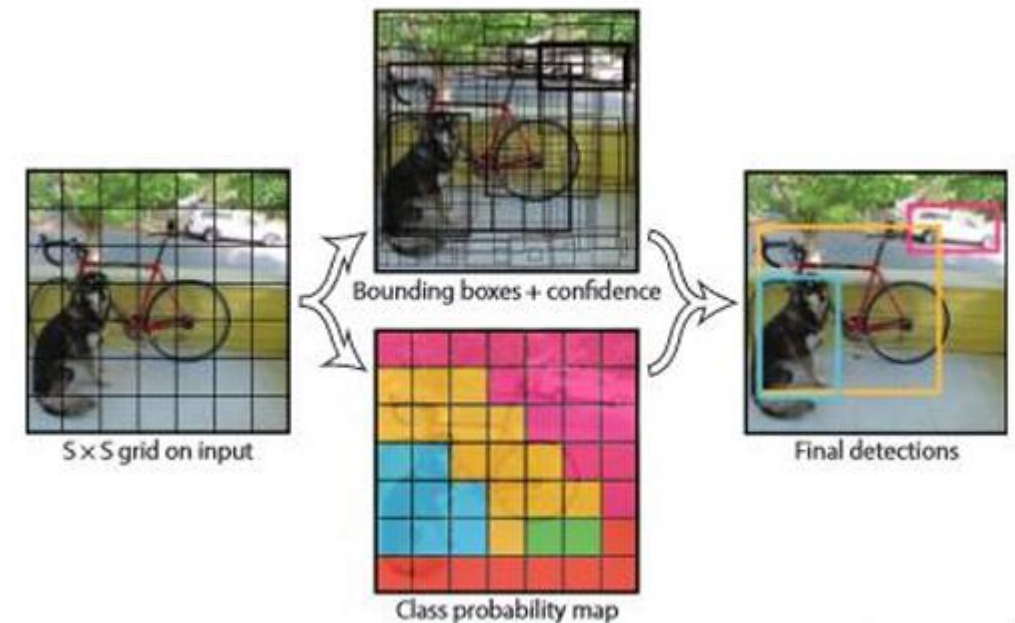
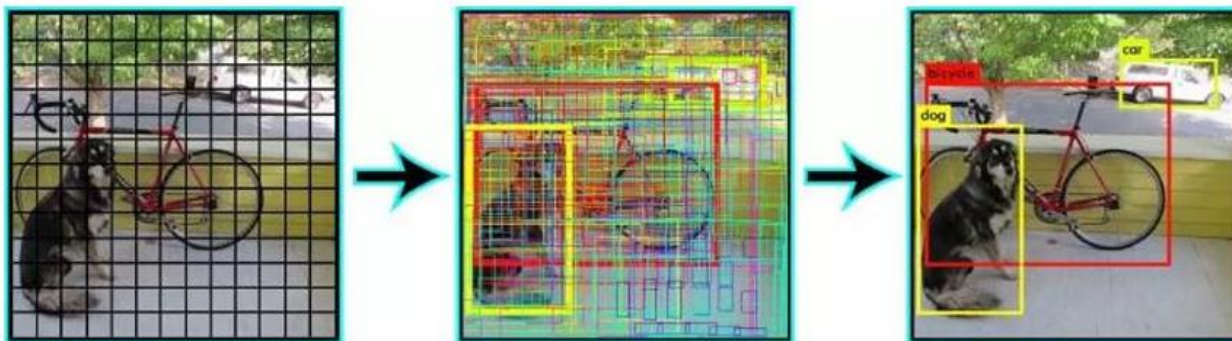


Bounding Box Regression:

Learn a transformation that maps the proposed box to the ground-truth box.

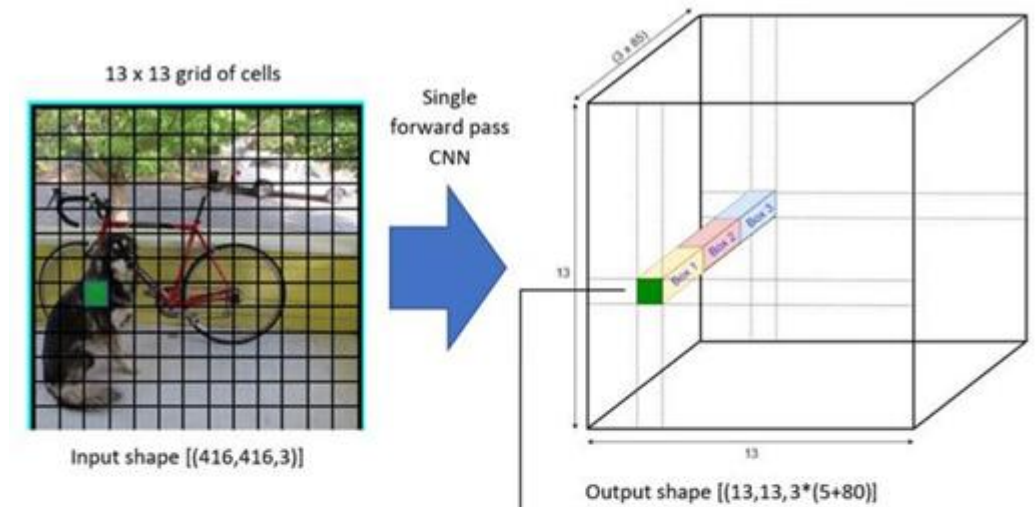
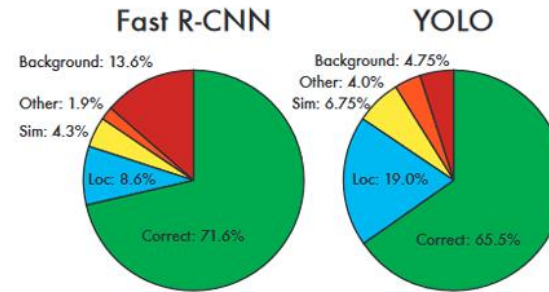
Object Detection - YOLO

- **You Only Look Once (YOLO):** is a fast, real-time object detector (45fps).
- It skips the region proposal step of R-CNN, only predicting over a limited number of bounding boxes (splits the image into $S \times S$ cells).
- Each cell predicts up to K bounding boxes:
 - (x, y) -offset from the center of the cell.
 - width and height of the bounding box.
 - object confidence.
 - one-hot encoded class prediction.



Object Detection - YOLO

Real-Time Detectors	Train	mAP	FPS
100Hz DPM [31]	2007	16.0	100
30Hz DPM [31]	2007	26.1	30
Fast YOLO	2007+2012	52.7	155
YOLO	2007+2012	63.4	45
Less Than Real-Time			
Fastest DPM [38]	2007	30.4	15
R-CNN Minus R [20]	2007	53.5	6
Fast R-CNN [14]	2007+2012	70.0	0.5
Faster R-CNN VGG-16[28]	2007+2012	73.2	7
Faster R-CNN ZF [28]	2007+2012	62.1	18
YOLO VGG-16	2007+2012	66.4	21



Type	Filters	Size	Output
Convolutional	32	3 × 3	256 × 256
Convolutional	64	3 × 3 / 2	128 × 128
1× Convolutional	32	1 × 1	
1× Convolutional	64	3 × 3	
1× Residual			128 × 128
2× Convolutional	128	3 × 3 / 2	64 × 64
2× Convolutional	64	1 × 1	
2× Convolutional	128	3 × 3	
2× Residual			64 × 64
8× Convolutional	256	3 × 3 / 2	32 × 32
8× Convolutional	128	1 × 1	
8× Convolutional	256	3 × 3	
8× Residual			32 × 32
8× Convolutional	512	3 × 3 / 2	16 × 16
8× Convolutional	256	1 × 1	
8× Convolutional	512	3 × 3	
8× Residual			16 × 16
4× Convolutional	1024	3 × 3 / 2	8 × 8
4× Convolutional	512	1 × 1	
4× Convolutional	1024	3 × 3	
4× Residual			8 × 8
Avgpool		Global	
Connected		1000	
Softmax			

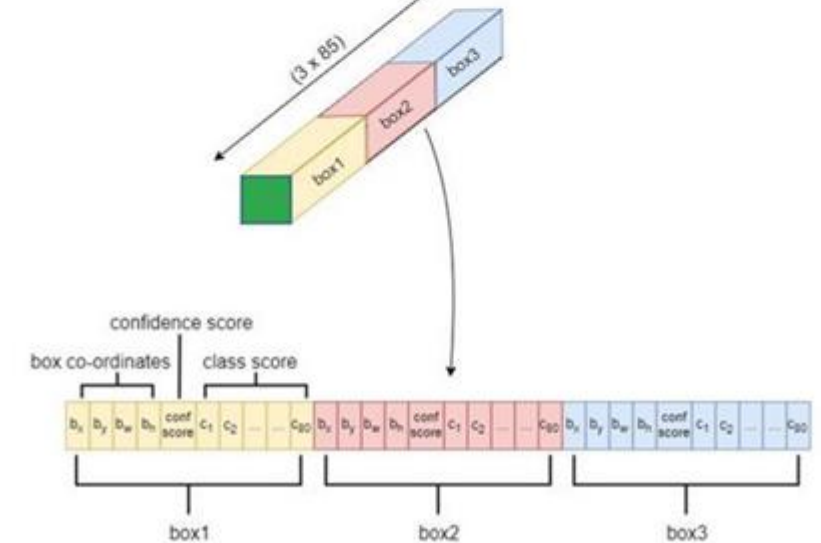
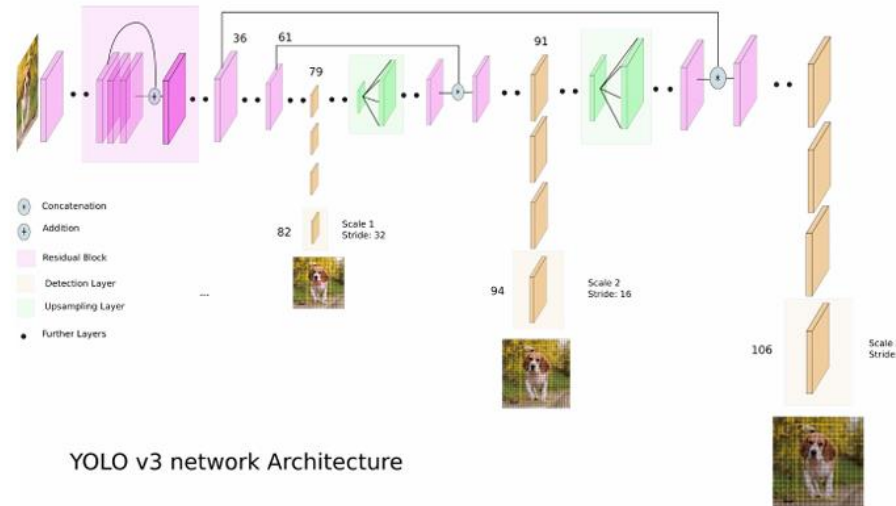
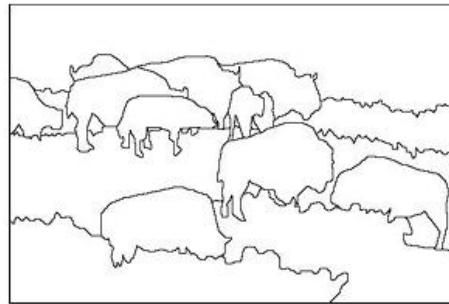


Image Segmentation

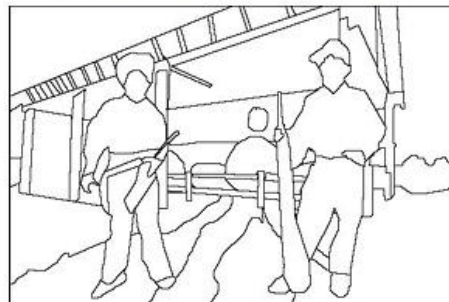
- **Image Segmentation** is the process of partitioning an image into distinct pixel-level segments (*semantic segmentation*: partition into distinct classes).
- The main goal of segmentation is to separate *foreground* from *background*.



image



human segmentation



(where to stop?)

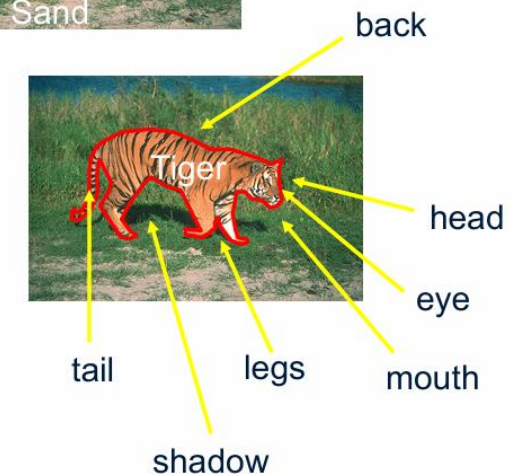
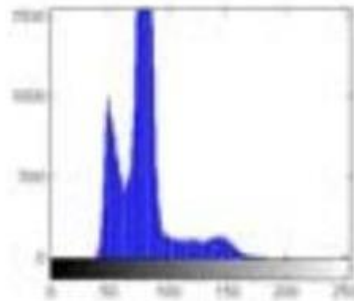
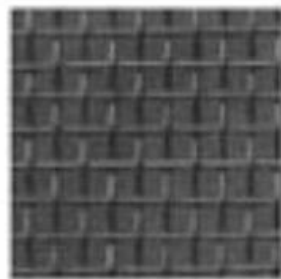


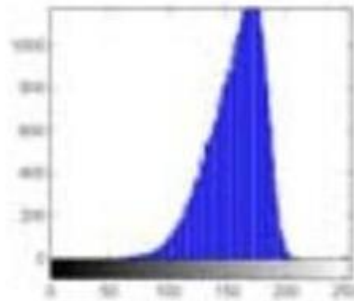
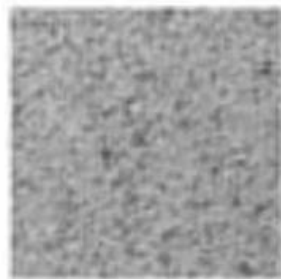
Image Segmentation - Texture

- Segmentation can be based on features such as intensity, color, position, **texture**, motion, depth, etc.
- The simplest way to describe texture is through *histogram moments*:



■ Brickwall texture

- Mean 79.13
- Variance 42.73
- CV 0.08
- Skew 1.37
- Kurtosis 5.93



■ Granite texture

- Mean 157.08
- Variance 96.9573
- CV 0.06
- Skew -0.73
- Kurtosis 3.25

Image Segmentation - Texture

- **Gray-Level Co-occurrence Matrix (GLCM)** texture features:

- $P(i, j, d)$: the probability of finding gray-level j when applying translation vector d to a pixel with a gray-level of i (most commonly $d = [1 \ 1]$).
- Measure how uniformly distributed the gray-level pairs are through *Entropy*:

$$Entropy = - \sum_i \sum_j P(i, j) \log P(i, j).$$

More statistics describing texture:

- Contrast

$$\sum_{i,j} |i - j|^2 P(i, j)$$

- Energy

$$\sum_{i,j} P(i, j)^2$$

- Homogeneity

$$\sum_{i,j} \frac{P(i, j)}{1 + |i - j|}$$

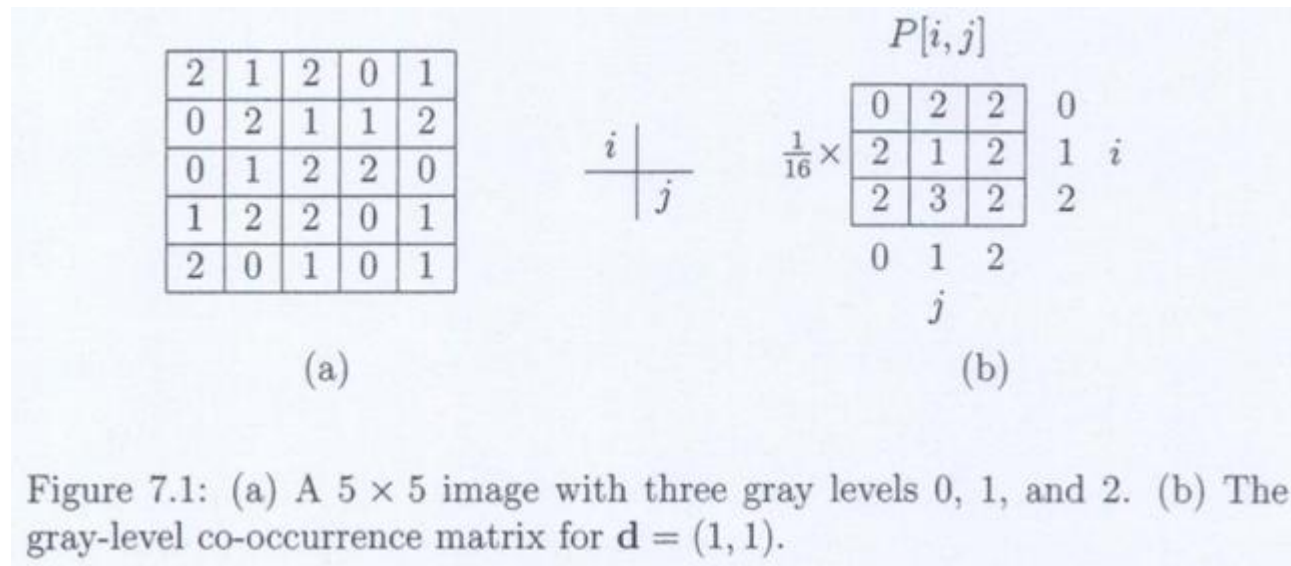


Image Segmentation - Texture

- **Autocorrelation Function** texture features:

- The autocorrelation function $R(k, l)$ of an image $g(m, n)$ is the sum of products of overlapping pixels as a patch is shifted over itself:

$$R(k, l) = \sum_m \sum_n g(m, n)g(m - k, n - l)$$

- *Coarse texture* → function drops off slowly.
- *Fine texture* → function drops off rapidly.
- *Periodic texture* → many peaks repeating far from (0,0).
- *Random texture* → only peak at (0,0).

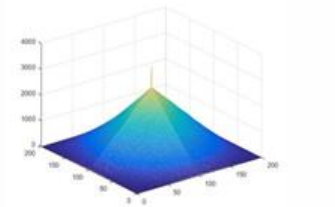
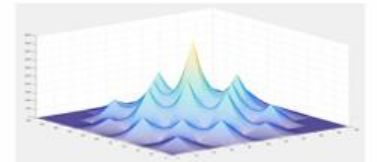
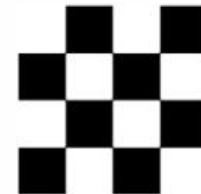
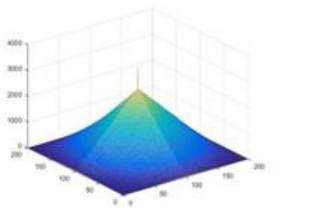
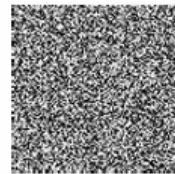
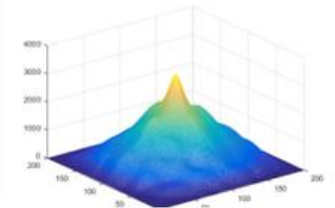


Image Segmentation - Texture

- **Texton** texture features:

1. Convolve the image with a bank of n filters, encoding each pixel as an n -d vector.
2. Perform clustering on the pixel vectors, each cluster is called a **Texton**.
3. The final texture feature is a histogram of textons computed on local neighborhoods.

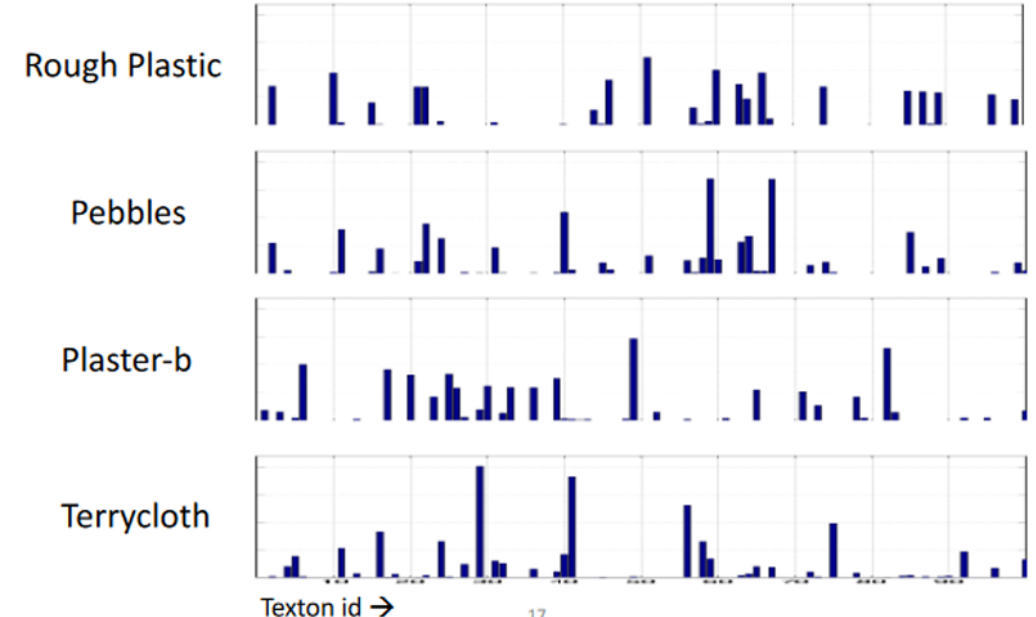
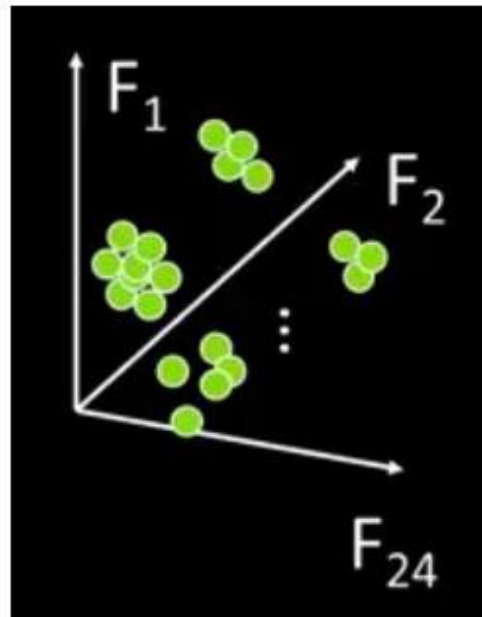
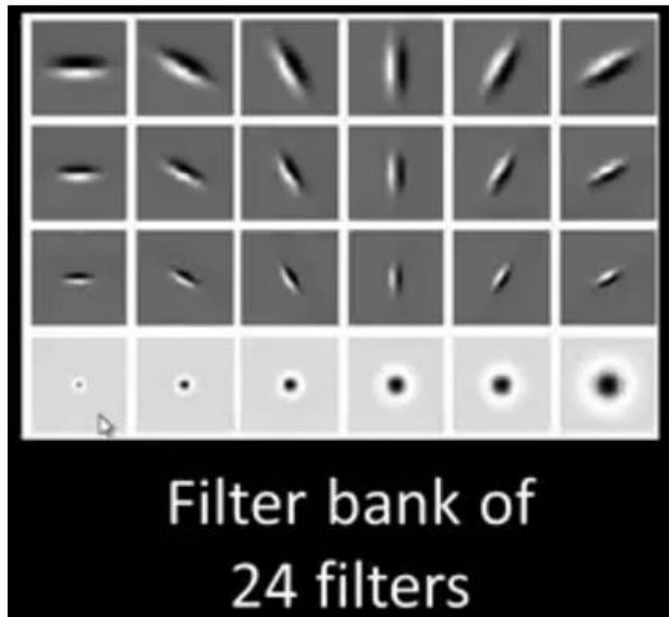


Image Segmentation - Thresholding

- **Thresholding** is the simplest way to apply segmentation, usually when objects are clearly distinguished from background or from each other.
- **Adaptive Thresholding** automatically determines a threshold T for every pixel based on the surrounding neighborhood.

- **Adaptive Mean Thresholding:**

Mean of neighborhood values.

- **Adaptive Gaussian Thresholding:**

Gaussian-weighted sum of neighborhood values.

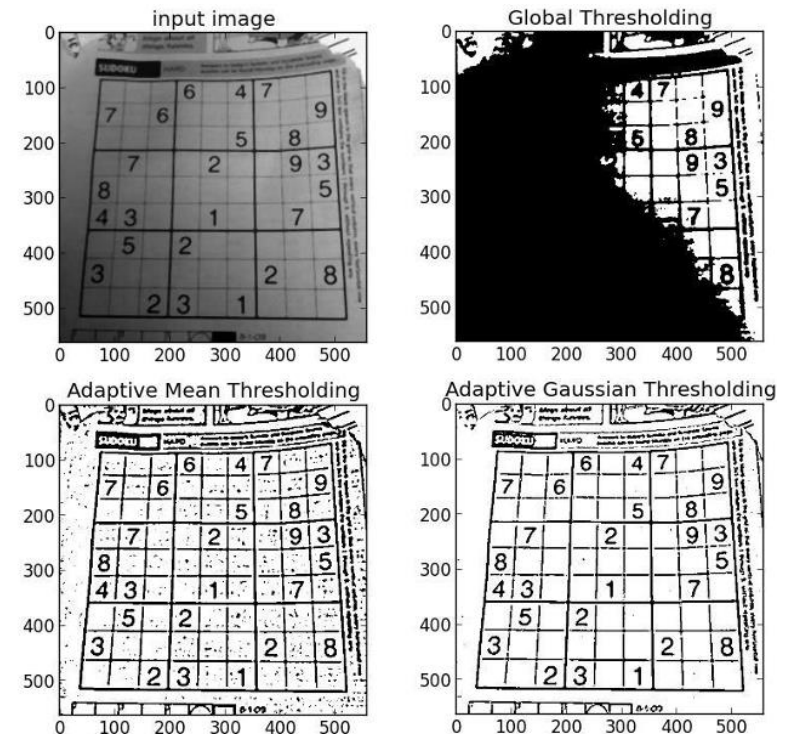


Image Segmentation - Watershed Algorithm

- The **Watershed Algorithm** is a fast segmentation algorithm, also used when objects are clearly distinguished from background or from each other.
 - Preferred algorithm for *hierarchical segmentation*.
 - It is often applied to the *gradient* image (edge map), for clearer separation.
1. Flood the topology from below, starting from local minima.
 2. When water from different basins is about to merge, build a dam.
 3. Finally, only the constructed dams are seen from above (*watershed lines*).

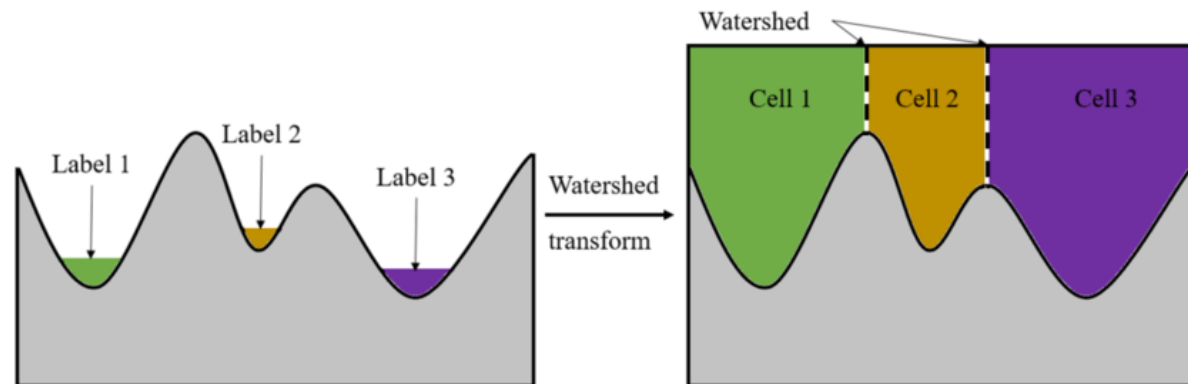


Image Segmentation - K-means Clustering

- **K-means Segmentation** using intensity or color $\rightarrow$ *Quantization* (there is no spacial coherence). To segment effectively, cluster based on color + position.
- K-means segmentation is guaranteed to converge to a local minimum, but:
 - Memory-intensive
 - Sensitive to initialization
 - Sensitive to outliers
 - Need to pick K beforehand
 - Only finds “spherical” clusters

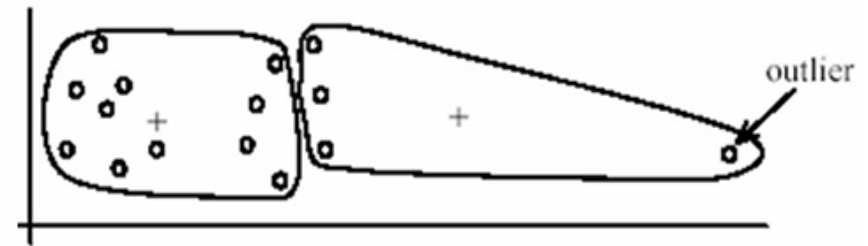
Image



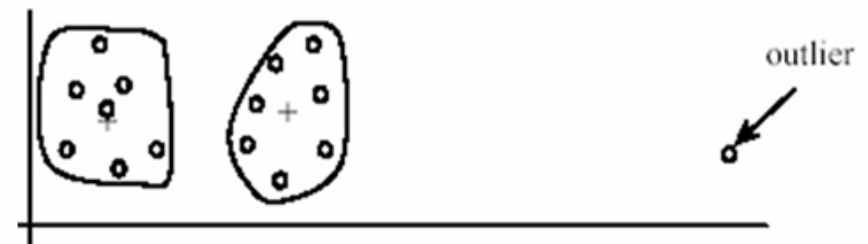
Intensity-based clusters



Color-based clusters



(A): Undesirable clusters



(B): Ideal clusters

Image Segmentation - Mean Shift Clustering

- The **Mean Shift** clustering algorithm looks for modes in a local (Gaussian) search window, shifting towards the center of mass until convergence.
- **Attraction Basin:** region for which all trajectories lead to the same mode. All data points in the attraction basin of a mode are a cluster.
- For segmentation:
 1. Find features.
 2. Initialize search windows at every feature point.
 3. Perform mean shift until convergence.
 4. Merge windows that end up at the same mode.
- Finds better clusters, is robust to outliers, and finds a variable no. of clusters.
- Computationally expensive and depends on the size of the search window.

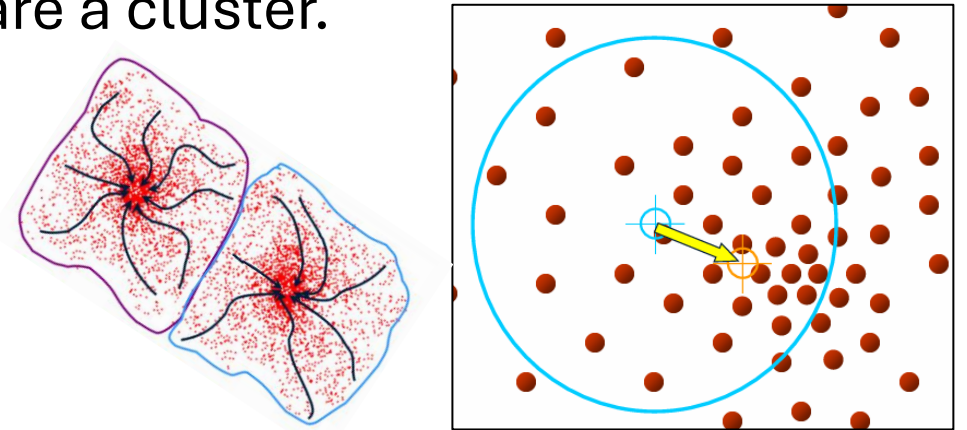
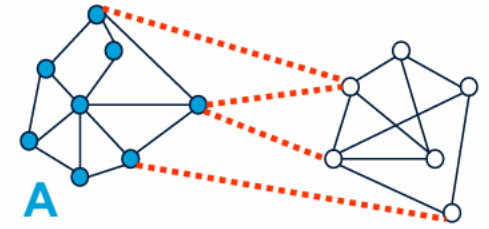


Image Segmentation - Mean Shift Clustering



Image Segmentation - Graph Cuts



- An image can be casted to a (complete) graph by treating every pixel as a node, with edges weighted by *affinity* (distance between feature vectors).
- A **Graph Cut** is a set of edges whose removal makes a graph disconnected.
- The *cost* of a cut is the sum of the weights of the cut edges.
- The simplest way to do segmentation is to find the **minimum cut** in the graph (efficient algorithms exist for this).
- The main downside of min-cut is that it tends to cut off small, isolated nodes.

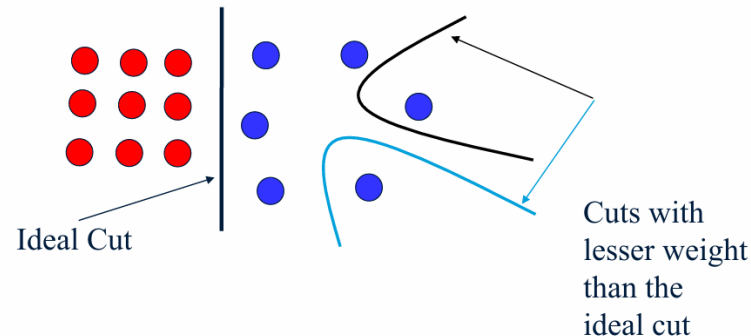


Image Segmentation - Graph Cuts

- A better way to cut the graph is by normalizing the cost by the weight of all edges connected to the associated nodes, known as **Normalized Cut**:

$$\text{normalized_cut_cost}(A, B) = \frac{e_{\{A,B\}}}{\sum_i e_{\{A,i\}}} + \frac{e_{\{A,B\}}}{\sum_i e_{\{B,i\}}}$$

- More efficient computation: $(D - W)y = \lambda Dy$, where W is the adjacency matrix of the graph, D is a diagonal matrix with $D(i, i) = \sum_j W(i, j)$, and y is the eigenvector corresponding to eigenvalue λ .
- Solve for the *second smallest* eigenvalue (the smallest is always 0, with an eigenvector of all 1s), for which the corresponding eigenvector can be used to *bipartition* the graph (assign nodes to group A or B based on their value in the eigenvector, usually with 0 as splitting point).

Image Segmentation - Graph Cuts

- Examples of recursively-applied normalized cut (with textons as features):



- Pros: general framework that can be used with many features.
- Cons: high computational and storage complexity, bias towards partitioning in equal segments.
- **Evaluation Criteria:** Jaccard Index (IoU) or F1 score.

$$Jacc(y, \hat{y}) = \frac{y \cap \hat{y}}{y \cup \hat{y}} = \frac{TP}{TP + FP + FN}$$

$$F1 = 2 \frac{\text{precision} \times \text{precision}}{\text{precision} + \text{precision}} = \frac{TP}{TP + \frac{1}{2}(FP + FN)}$$

Image Segmentation - FCN

- A **Fully Convolutional Network (FCN)** learns a mapping from pixel to pixel without needing to extract region proposals (*semantic segmentation*).
- Compared to a CNN, an FCN replaces the FC layers with convolutional layers and converts the final (output) layer to the original image resolution.

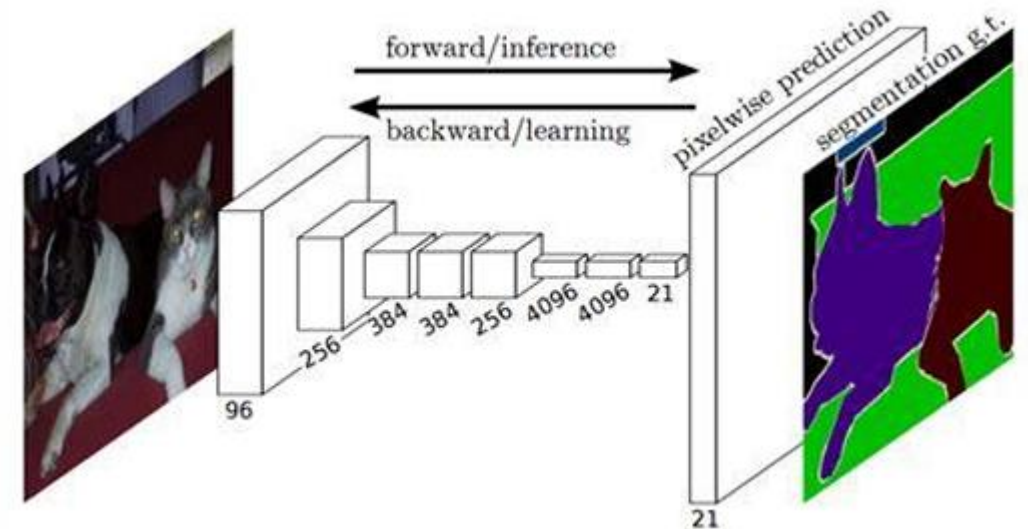


Image Segmentation - FCN

- **Dilated Convolutions** support the expansion of the receptive field from 3x3 to 7x7 to 15x15 without loss of resolution or adding new parameters.
- It works by inserting gaps between its elements, “skipping” certain pixels.

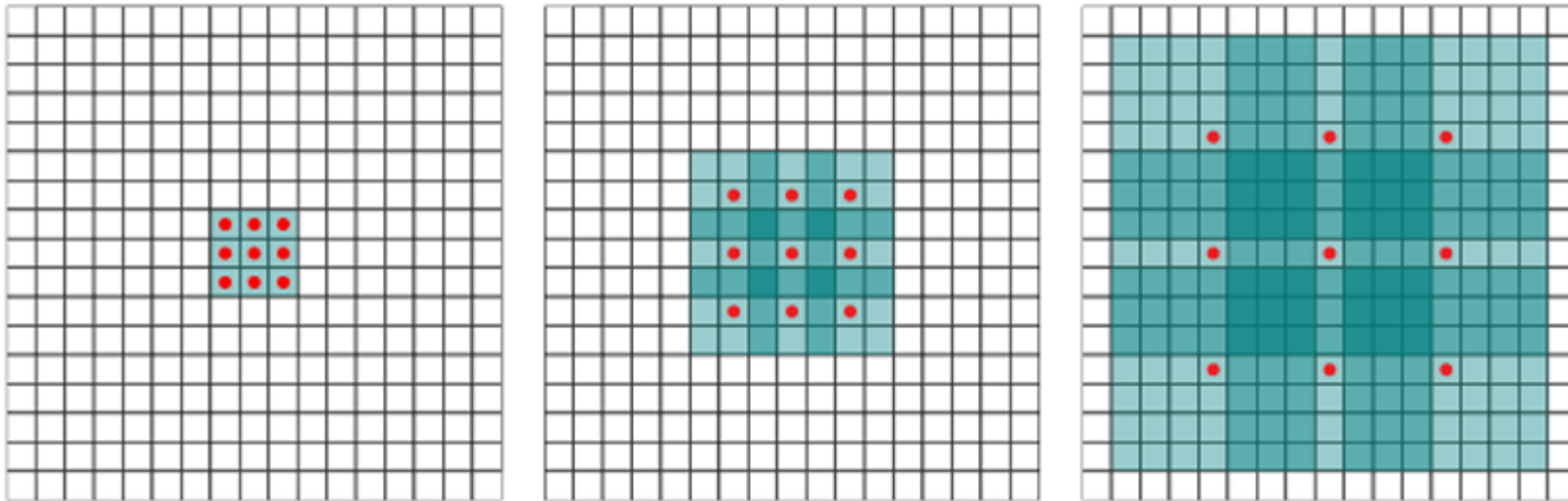


Image Segmentation - SegNET

- Efficient and lightweight FCN segmentation model.
- Re-using max pool indices from the encoder for upsampling (not learned).

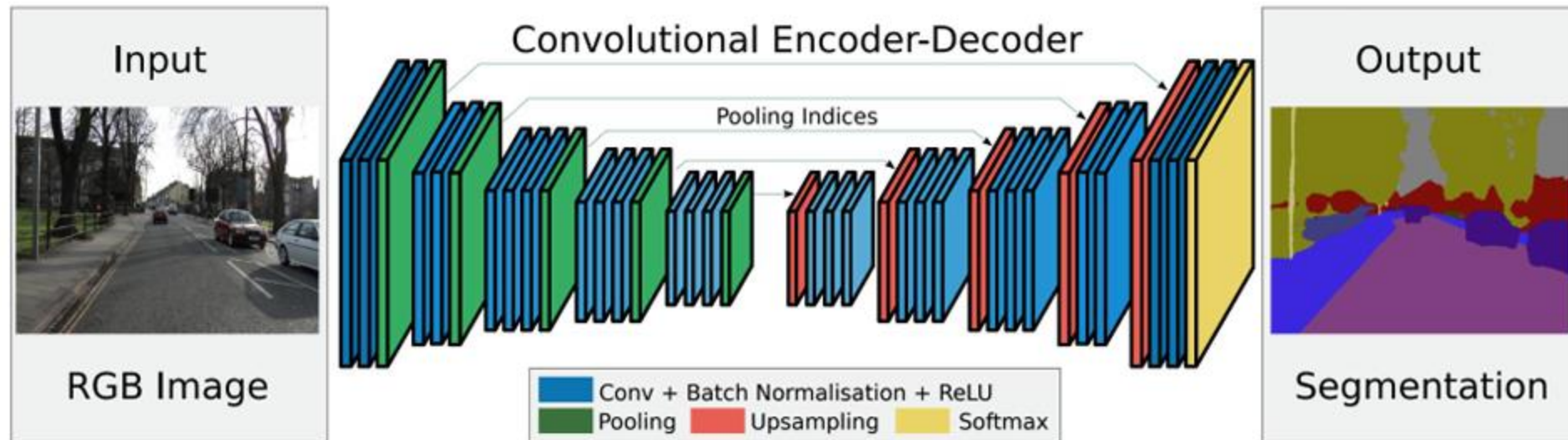
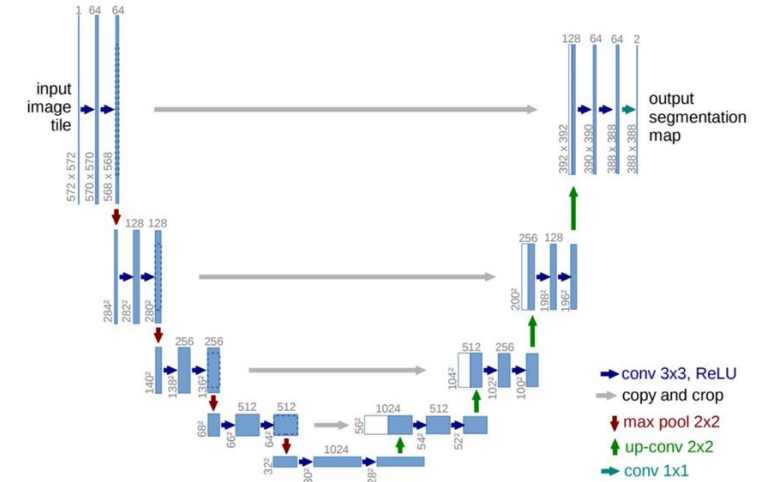


Image Segmentation - U-NET

- High accuracy and detailed FCN segmentation model.
- Uses skip connections and learned upsampling.



Ground Truth
Semantic Map

Generated
Semantic Map
(Model 1)

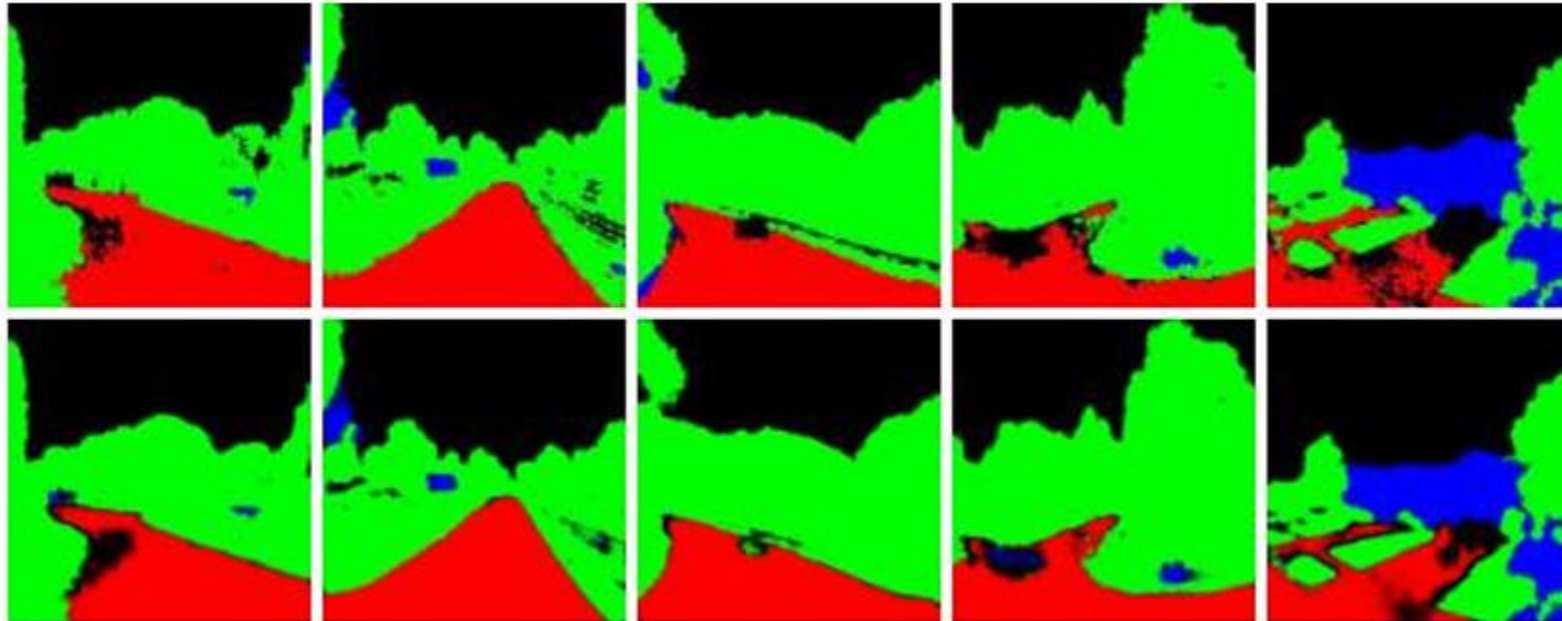


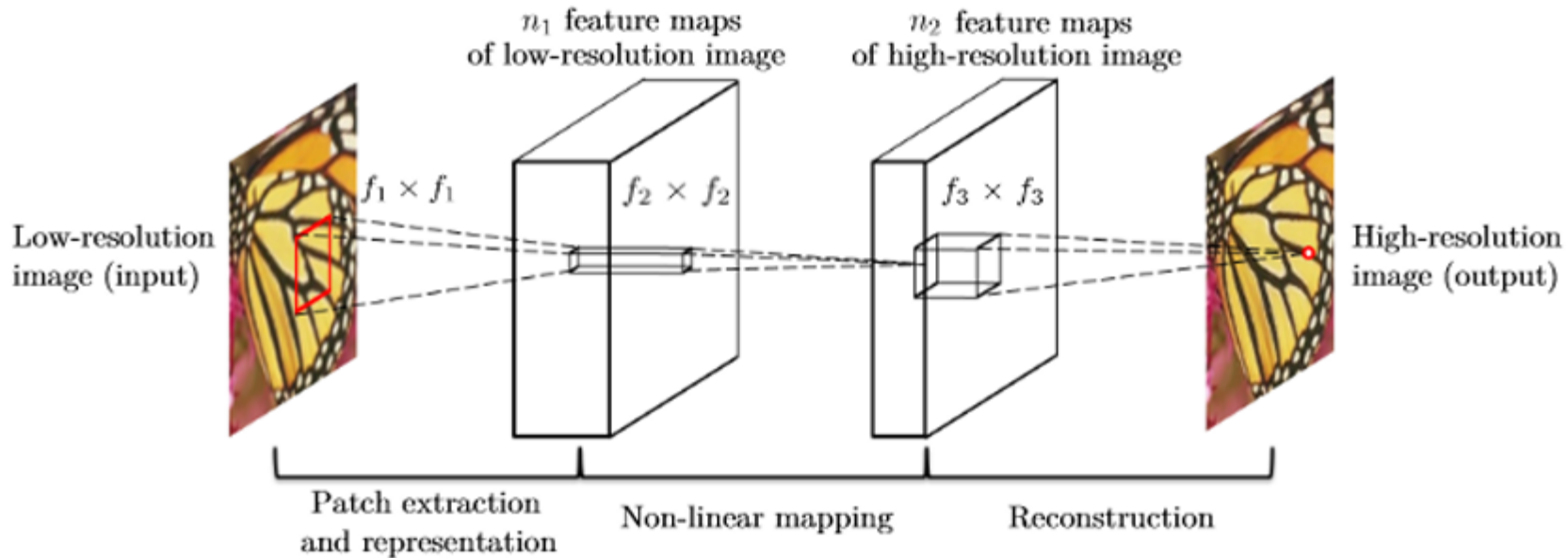
Image Segmentation - SAM

- **Segment Anything Model (SAM)** is a promptable segmentation model with zero-shot generalization capabilities.



Image Upscaling

- **Super-Resolution CNN (SRCNN)** is a lightweight CNN for mapping between low- and high-resolution images.



Neural Style Transfer

- **Style Transfer** is a technique for recomposing images in the style of other images.
- This is done using CNNs that learn to find separate representations of the *content* and the *style* of an image.
- Loss function: $\mathcal{L}_{total} = \alpha \mathcal{L}_{content} + \beta \mathcal{L}_{style}$

